BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No: 4

Roll no: 53 PRN No: 12414903 Name of Student: **Sara Ansingkar**

Subject Name : Deep Learning

Problem Statement :

Develop an LSTM-based model for time-series forecasting using stock price, weather, or sales datasets.

Algorithm :

1. Start
2. Import the required libraries such as NumPy, Pandas, Matplotlib, Scikit-learn, and TensorFlow/Keras.
3. Load the stock-price dataset containing historical data.
4. Select the Closing Price (Close) column as the target variable.
5. Sort the data according to the date/time to maintain the correct time-series order.
6. Select a suitable number of records from the dataset to reduce computation time.
7. Normalize the closing-price values using Min-Max Scaling so that the values lie between 0 and 1.
8. Create input sequences using a fixed time window:
    - Use the previous 60 days of closing prices as input.
    - Use the following day's closing price as the target.
9. Divide the sequences into training and testing datasets while maintaining chronological order.
10. Reshape the input data into the 3D format required by LSTM:

- Samples × Time Steps × Features.

11. Build the LSTM model using:

- LSTM layer
- Dropout layer to reduce overfitting
- Dense output layer

12. Compile the model using the Adam optimizer and Mean Squared Error loss.
13. Train the model on the training data for a specified number of epochs.

14. Use the trained model to predict stock prices for the test dataset.
15. Apply inverse scaling to convert the predicted values back to the original price scale.
16. Calculate Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE) to evaluate the model.
17. Plot the actual vs predicted stock prices.

Code and Output:

```
[ ]
# Problem Statement:
# Develop an LSTM-based model for time-series forecasting
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
```

```
[ ]
import kagglehub
path = kagglehub.dataset_download("vidhisrivastava/weather-dataset")

print("Path to dataset files:", path)
```

```
Downloading to /root/.cache/kagglehub/datasets/vidhisrivastava/weather-dataset/1.archive...
100%|██████████| 101k/101k [00:00<00:00, 33.1MB/s]Extracting files...
Path to dataset files: /root/.cache/kagglehub/datasets/vidhisrivastava/weather-dataset/versions/1
```

```
[3]
Os
print(os.listdir(path))
```

```
['Project1WeatherDataset.csv']
```

```
[4]
Os
file_path = os.path.join(path, "Project1WeatherDataset.csv")

df = pd.read_csv(file_path)

print(df.head())
print("Original shape:", df.shape)
print(df.columns)
```

```
        Date/Time  Temp_C  Dew Point Temp_C  Rel Hum_%  Wind Speed_km/h  \
0  1/1/2012 0:00    -1.8              -3.9         86                4
1  1/1/2012 1:00    -1.8              -3.7         87                4
2  1/1/2012 2:00    -1.8              -3.4         89                7
3  1/1/2012 3:00    -1.5              -3.2         88                6
4  1/1/2012 4:00    -1.5              -3.3         88                7
```

```
   Visibility_km  Press_kPa              Weather
0            8.0     101.24                  Fog
1            8.0     101.24                  Fog
2            4.0     101.26  Freezing Drizzle,Fog
3            4.0     101.27  Freezing Drizzle,Fog
4            4.8     101.23                  Fog
Original shape: (8784, 8)
Index(['Date/Time', 'Temp_C', 'Dew Point Temp_C', 'Rel Hum_%',
       'Wind Speed_km/h', 'Visibility_km', 'Press_kPa', 'Weather'],
      dtype='object')
```

[5] ✓ 0s

```
df = df.head(1000).copy()

print("New dataset shape:", df.shape)
```

```
New dataset shape: (1000, 8)
```

[6] ✓ 0s

```
print(df.isnull().sum())
print(df.describe())
```

```
Date/Time           0
Temp_C              0
Dew Point Temp_C    0
Rel Hum_%           0
Wind Speed_km/h     0
Visibility_km       0
Press_kPa           0
Weather             0
dtype: int64
            Temp_C  Dew Point Temp_C    Rel Hum_%  Wind Speed_km/h  \
count  1000.000000       1000.000000  1000.000000      1000.000000
mean     -6.792300        -11.742500    68.441000        17.277000
std       6.076123          6.835978    11.106299         9.994206
min     -23.300000        -28.500000    39.000000         0.000000
25%     -11.200000        -16.200000    60.000000         9.000000
50%      -6.300000        -11.850000    69.000000        17.000000
75%      -1.800000         -6.600000    78.000000        22.000000
max       5.300000          3.600000    94.000000        83.000000

       Visibility_km    Press_kPa
count    1000.000000  1000.000000
mean       22.615800   101.113710
std        11.442274     0.965991
min         0.600000    97.990000
25%        16.100000   100.530000
50%        25.000000   101.130000
75%        25.000000   101.880000
max        48.300000   103.090000
```

```
df["Date/Time"] = pd.to_datetime(df["Date/Time"], errors="coerce")

df = df.sort_values("Date/Time")

df = df.dropna(subset=["Date/Time"])

print(df[["Date/Time"]].head())
```

```
            Date/Time
0 2012-01-01 00:00:00
1 2012-01-01 01:00:00
2 2012-01-01 02:00:00
3 2012-01-01 03:00:00
4 2012-01-01 04:00:00
```

```
temperature = df[["Temp_C"]].values

print(temperature[:5])
print("Number of records:", len(temperature))
```

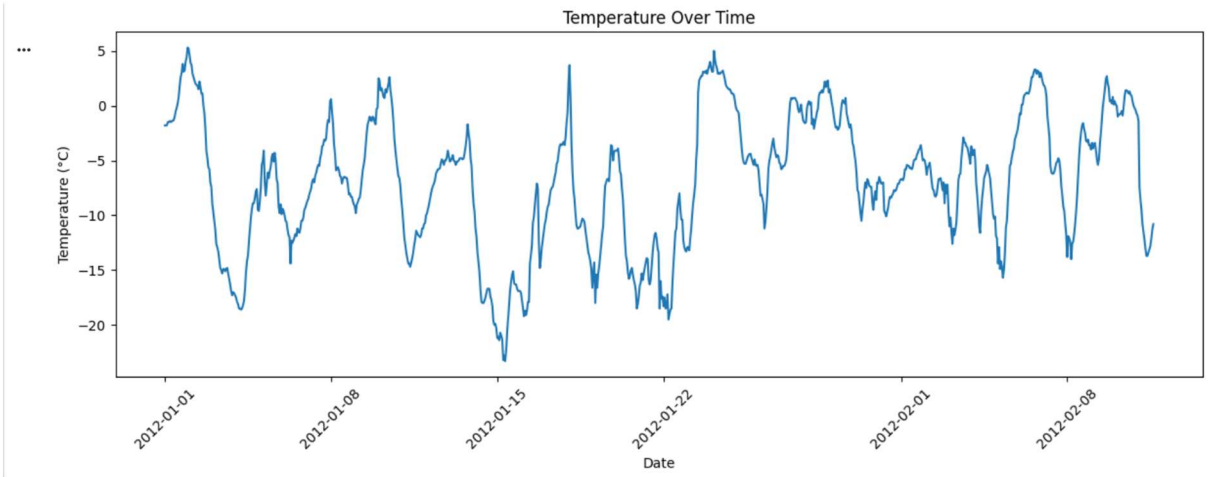
```
[[-1.8]
 [-1.8]
 [-1.8]
 [-1.5]
 [-1.5]]
Number of records: 1000
```

```
plt.figure(figsize=(12, 5))

plt.plot(df["Date/Time"], df["Temp_C"])

plt.xlabel("Date")
plt.ylabel("Temperature (°C)")
plt.title("Temperature Over Time")

plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



[10]
✓ 0s

```
scaler = MinMaxScaler(feature_range=(0, 1))

scaled_temperature = scaler.fit_transform(temperature)

print(scaled_temperature[:5])
```

```
[[0.75174825]
 [0.75174825]
 [0.75174825]
 [0.76223776]
 [0.76223776]]
```

[11]
✓ 0s

```
sequence_length = 24

X = []
y = []

for i in range(sequence_length, len(scaled_temperature)):
    X.append(scaled_temperature[i-sequence_length:i])
    y.append(scaled_temperature[i])

X = np.array(X)
y = np.array(y)

print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
X shape: (976, 24, 1)
y shape: (976, 1)
```

```
train_size = int(len(X) * 0.8)

X_train = X[:train_size]
X_test = X[train_size:]

y_train = y[:train_size]
y_test = y[train_size:]

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

```
Training samples: 780
Testing samples: 196
```

```
model = Sequential([
    LSTM(32, input_shape=(X_train.shape[1], X_train.shape[2])),
    Dropout(0.2),
    Dense(1)
])

model.compile(
```

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 32)	4,352
dropout (Dropout)	(None, 32)	0
dense (Dense)	(None, 1)	33

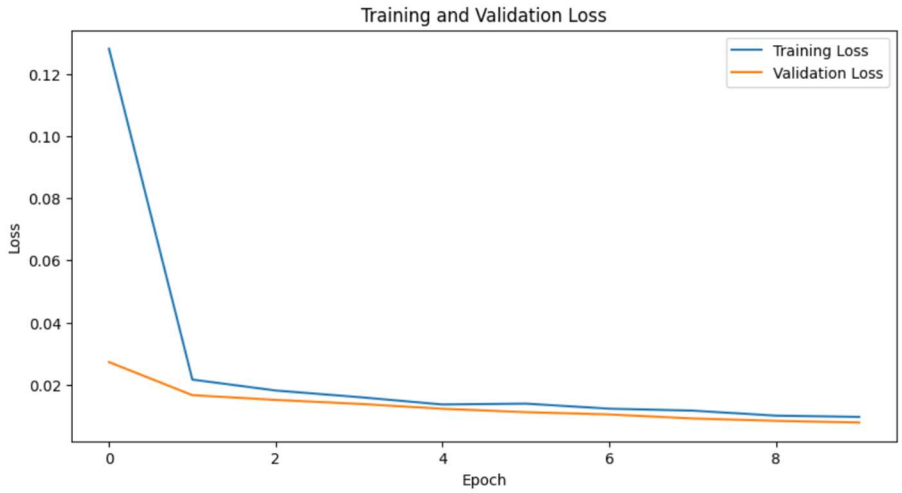
Total params: 4,385 (17.13 KB)
Trainable params: 4,385 (17.13 KB)
Non-trainable params: 0 (0.00 B)

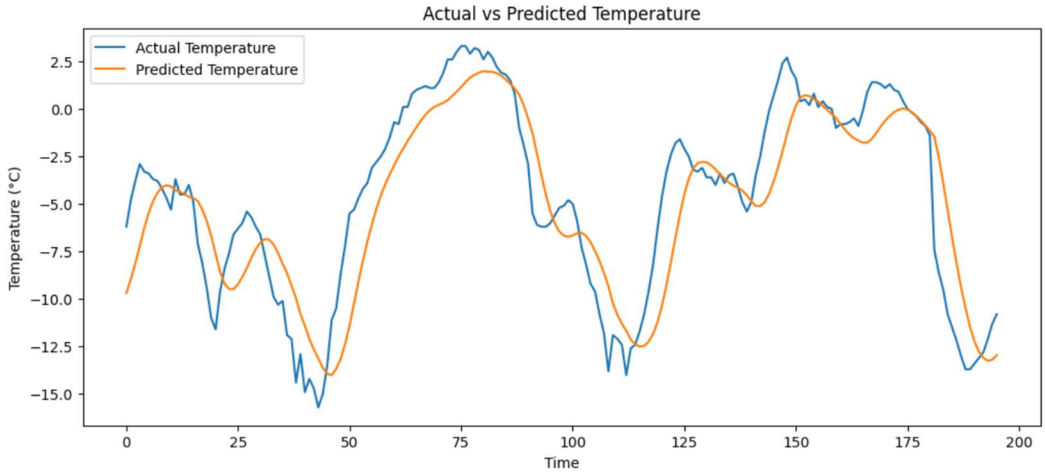
+ Code + Text

[14] ✓ 4s

```
history = model.fit(
    X_train,
    y_train,
    epochs=10,
    batch_size=32,
    validation_data=(X_test, y_test),
    verbose=1
)
```

```
Epoch 1/10
25/25 ━━━━━━━━━━━━━━━━━━━━ 2s 19ms/step - loss: 0.1280 - val_loss: 0.0273
Epoch 2/10
25/25 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0216 - val_loss: 0.0166
```


Training and Validation Loss
Training Loss
Validation Loss
Loss
0.12
0.10
0.08
0.06
0.04
0.02
0
2
4
6
8
Epoch


Actual vs Predicted Temperature
Actual Temperature
Predicted Temperature
Temperature (°C)
2.5
0.0
−2.5
−5.0
−7.5
−10.0
−12.5
−15.0
0
25
50
75
100
125
150
175
200
Time